Лабораторная работа №11

Тема: Разработка триггеров

Цель: получить практические навыки реализации специфических ограничений определения структур данных и манипулирования данными при помощи триггеров.

Объем: 4 академических часа (1 неделя).

Задание

1. Проанализировать схему базы данных, разработанной при выполнении лабораторных работ №5 и №6, и определить для отслеживания несколько DML-событий, не менее 1 события каждого типа.
2. Для определенных в п.1 событий разработать команды SQL для создания соответствующих DML-триггеров типов BEFORE. Выполнить команды и убедиться в корректности их выполнения.
3. В соответствии с индивидуальным заданием согласно табл. 11.1 создать DML-триггеры типа AFTER для логирования манипуляций с оперативными данными БД. Для этого создать таблицу logs, в которой определить поле date, где будет записываться дата и время (timestamp) срабатывания триггера, и поле info, где будет записываться информация о вызванной команде.
4. Добавить в любой из справочников БД поле IsDeleted типа Boolean (по умолчанию false) и создать для этого справочника DML-триггер для реализации мягкого удаления данных - при выполнении удаления, для тех строк, где IsDeleted стоит false триггер устанавливает это поле как true, вместо того чтобы их удалить, а при повторном вызове уже удаляет.
5. В соответствии с индивидуальными заданием согласно табл. 11.1 разработать DDL-триггер (event trigger) для блокировки в БД DDL-событий. Предусмотреть вывод сообщения о блокировке, содержащего имя БД.

6. Выполнить тестирование разработанных в пп. 2–5 триггеров посредством инициирования в БД соответствующих событий. Зафиксировать результаты срабатывания триггеров скриншотами.
7. Доработать приложение, разработанное при выполнении лабораторных работ №9 и №10, обеспечить правильную обработку исключений при получении ошибок от срабатывания триггеров.

Таблица 11.1. Индивидуальные задания на создание триггеров

№	DML-триггеры типа AFTER	DDL-триггеры
1	INSERT	ALTER_TABLE
2	UPDATE	DROP_TABLE
3	UPDATE	ALTER_VIEW
4	DELETE	DROP_VIEW
5	INSERT	ALTER_TRIGGER
6	UPDATE	CREATE_VIEW
7	UPDATE	ALTER_TABLE
8	DELETE	DROP_TABLE
9	INSERT	ALTER_VIEW
10	UPDATE	DROP_VIEW
11	UPDATE	ALTER_TRIGGER
12	DELETE	CREATE_VIEW
13	INSERT	ALTER_TABLE
14	UPDATE	DROP_TABLE
15	UPDATE	ALTER_VIEW
16	DELETE	DROP_VIEW
17	INSERT	ALTER_TRIGGER
18	UPDATE	CREATE_TRIGGER
19	UPDATE	ALTER_TABLE
20	DELETE	DROP_TABLE

Методические указания к выполнению

Триггеры представляют собой специальный тип хранимой процедуры, которая вызывается автоматически при выполнении определенного действия над таблицей или представлением, в частности, при добавлении, изменении или удалении данных, то есть при выполнении команд INSERT, UPDATE, DELETE. Количество триггеров в таблице не ограничивается.

Формальное определение DML триггера:

```
CREATE [ OR REPLACE ] TRIGGER name { BEFORE | AFTER | INSTEAD OF } { event [ OR ... ] }
    ON table_name
    [ FOR [ EACH ] { ROW | STATEMENT } ]
    [ WHEN ( condition ) ]
    EXECUTE { FUNCTION | PROCEDURE } function_name ( arguments )
```

table_name – имя таблицы для которой устанавливается триггер.

Перед созданием самого триггера, необходимо перед этим выше определить функцию либо процедуру function_name, которую он будет вызывать при срабатывании, при этом она обязательно должна возвращать тип данных trigger или event_trigger.

В теле функции будут доступны специальные ключевые слова, через которые можно получить доступ к некоторым объектам. Они доступны только в триггерных функциях:

NEW - Переменная содержит новую строку базы данных для команд INSERT/UPDATE в триггерах уровня строки. В триггерах уровня оператора и для команды DELETE этой переменной значение не присваивается.

OLD - Переменная содержит старую строку базы данных для команд UPDATE/DELETE в триггерах уровня строки. В триггерах уровня оператора и для команды INSERT этой переменной значение не присваивается.

TG_NAME - Переменная содержит имя сработавшего триггера.

TG_OP - Строка, содержащая INSERT, UPDATE, DELETE или TRUNCATE, в зависимости от того, для какой операции сработал триггер.

TG_TABLE_NAME - Имя таблицы, для которой сработал триггер.

TG_TABLE_SCHEMA - Имя схемы, содержащей таблицу, для которой сработал триггер.

Кроме того, в тело функции, которая будет использоваться для триггера можно включить команды обработки исключений:

- RAISE EXCEPTION позволяет отправить сообщение об критической ошибке (и прервать выполнение).
- RAISE NOTICE отправляет пользователю уведомление (просто сообщение, не ошибка, выполнение не прерывается).

Тип (фаза работы) триггера:

- BEFORE (срабатывает до изменения данных);
- AFTER (срабатывает после изменения данных);
- INSTEAD OF (срабатывает вместо указанного события, можно определять только для представлений).

Тип модификации данных, приводящей к срабатыванию триггера:

INSERT | UPDATE | DELETE | TRUNCATE. При создании триггера можно задать одну, две или три операции. В случае выбора нескольких операций их следует разделить через or: INSERT OR UPDATE OR DELETE.

Отключение/включение триггера:

```
ALTER EVENT TRIGGER trigger_name DISABLE;
ALTER EVENT TRIGGER trigger_name ENABLE;
```

Изменение триггера:

```
ALTER TRIGGER trigger_name …
```

Удаление триггера:

```
DROP TRIGGER trigger_name
```

Триггеры типа BEFORE

Рассмотрим пример триггера типа BEFORE, запрещающего добавление в БД данных о наборах более позднего года, чем текущий:

```
CREATE OR REPLACE FUNCTION check_year() RETURNS trigger AS
$check_year$
   BEGIN
      IF NEW."setyear" IS NULL THEN
         RAISE EXCEPTION 'укажите год!';
      END IF;
      IF NEW."setyear" > date_part('year', now()) THEN
         RAISE EXCEPTION 'Нельзя добавлять наборы позднее текущего
года';
      END IF;
      RETURN NEW;
   END;
$check_year$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER check_year BEFORE INSERT OR UPDATE
ON "sets"
   FOR EACH ROW EXECUTE PROCEDURE check_year();
```

При попытке добавления или изменения, любым способом: через вызов команды, через PgAdmin, либо через запрос из приложения на C#, вызванная команда не должна сработать, вместо этого мы получаем вызов исключения с сообщением об ошибке. Конечно, в случае приложения на C# нам ещё нужно эту ошибку правильно обработать.

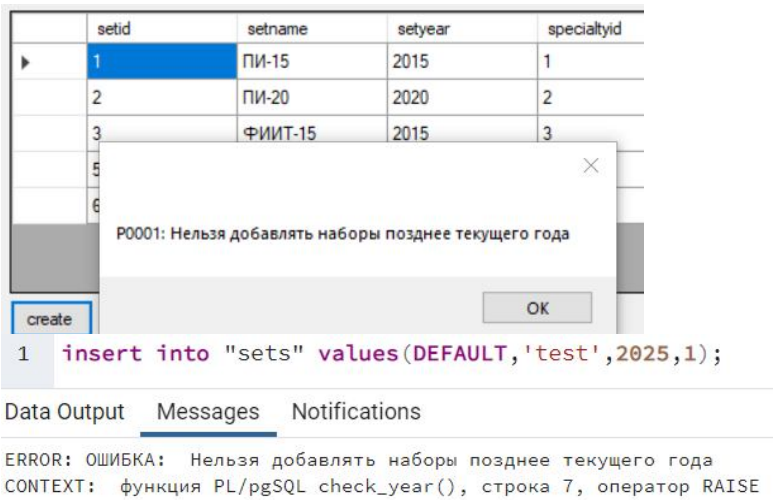

Рисунок 11.1 – Срабатывание триггера

Пример триггера, реализующего мягкое удаление для таблицы групп. Сначала флаг isdeleted устанавливается на true, и только после второго вызова команды, запись удаляется из таблицы:

```
CREATE OR REPLACE FUNCTION delete_group() RETURNS trigger AS $delete_group$
        DECLARE
    command text := '" SET "isdeleted" = true WHERE "groupid" = $1';
  BEGIN
        IF (OLD."isdeleted" IS NULL) or (OLD."isdeleted" = false) THEN
      EXECUTE 'UPDATE "' || TG_TABLE_NAME || command USING OLD."groupid";
RETURN NULL; END IF;
RETURN OLD; END;
$delete_group$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER delete_group
BEFORE DELETE ON "groups"
FOR EACH ROW EXECUTE PROCEDURE delete_group();
```

Триггеры типа AFTER

Триггеры AFTER активизируется после успешного выполнения всех операций, указанных в запускающем операторе или операторах SQL. Сюда включается весь каскад действий по ссылкам и все проверки ограничений. Триггеры AFTER используются для учета (архивирования) выполненных операций, либо проверки постусловий исполнения команды.

Рассмотрим пример триггера типа AFTER, который записывает лог выполнения трех типов команд над таблицей наборов:

```
CREATE OR REPLACE FUNCTION log_sets() RETURNS TRIGGER AS $log_sets$
    BEGIN
        IF (TG_OP = 'DELETE') THEN
            INSERT INTO "logs" SELECT now(), 'DELETE';
            RETURN OLD;
        ELSIF (TG_OP = 'UPDATE') THEN
            INSERT INTO "logs" SELECT now(), 'UPDATE';
            RETURN NEW;
        ELSIF (TG_OP = 'INSERT') THEN
            INSERT INTO "logs" SELECT now(), 'INSERT';
            RETURN NEW;
        END IF;
        RETURN NULL;
    END;
$log_sets$ LANGUAGE plpgsql;

CREATE TRIGGER log_sets
AFTER INSERT OR UPDATE OR DELETE ON "sets"
    FOR EACH ROW EXECUTE PROCEDURE log_sets();
```

При выполнении любой из трех команд над таблицей наборов, и при успешном их завершении, срабатывает триггер и записывает тип команды и текущие дату/время в таблицу учета для архивирования.

	date timestamp with time zone	info text
1	2024-02-26 23:18:38.598278+03	INSERT
2	2024-02-26 23:18:43.262341+03	UPDATE
3	2024-02-26 23:18:45.011222+03	DELETE

DDL-Триггеры

Триггеры DDL (в PostgresSQL они называются event_trigger) могут быть использованы в административных задачах, таких как аудит и регулирование операций базы данных. Действие распространяется на все команды одного типа во всей БД или на всём сервере. Типовые задачи для DDL-триггеров:

- предотвратить внесение определенных изменений в схему БД;
- выполнить в БД некоторые действия в ответ на изменения в схеме;
- записывать изменения или события схемы БД;

Формальное определение триггера:

```
CREATE EVENT TRIGGER name ON event
[ WHEN filter_variable IN (filter_value [, ... ]) [ AND ... ] ]
EXECUTE { FUNCTION | PROCEDURE } function_name()
```

Такие триггеры ставятся на глобальные события:

ddl_command_start, ddl_command_end, table_rewrite и sql_drop.

Событие ddl_command_start происходит непосредственно перед выполнением команд CREATE, ALTER, DROP, SECURITY LABEL, COMMENT, GRANT и REVOKE. Проверка на существование объекта перед срабатыванием триггера не производится. Событие ddl_command_start также происходит непосредственно перед выполнением команды SELECT INTO, так как она равнозначна команде CREATE TABLE AS.

В определении триггера можно использовать условие WHEN, чтобы, например, триггер ddl_command_start срабатывал только для отдельных команд, которые нужно перехватить, тогда в качестве filter_variable нужно использовать встроенную переменную TAG, а в качестве значений набор названий команд, например, (‘CREATE TABLE’).

Событие ddl_command_end происходит непосредственно после выполнения команд из того же набора. Чтобы получить дополнительную информацию об операциях DDL, повлёкших произошедшее событие, можно вызовать функцию pg_event_trigger_ddl_commands().

Событие table_rewrite происходит только после того, как таблица будет перезаписана в результате определённых действий команды ALTER TABLE. Если есть несколько триггеров на одно и то же событие, то они будут вызываться в алфавитном порядке по имени триггера.

Как и в случае с DML триггерами, для DDL предварительно нужно создать функцию, со специальным возвращаемым типом event_trigger. Данная функция не обязана возвращать значение (и может не возвращать). Возвращаемый тип служит лишь указанием на то, что функция будет вызываться из триггера события.

Пример создания триггера, который запрещает выполнение команды на создание таблицы:

```
CREATE OR REPLACE FUNCTION abort_create_table_command()
  RETURNS event_trigger
 LANGUAGE plpgsql
  AS $$
BEGIN
  RAISE EXCEPTION 'команда % отключена', tg_tag;
END;
$$;

CREATE EVENT TRIGGER abort_create_table ON ddl_command_start
        WHEN TAG IN ('CREATE TABLE')
        EXECUTE PROCEDURE abort_create_table_command();
```

```
1   CREATE TABLE IF NOT EXISTS public."test" (id integer)
```

Data Output Messages Notifications

```
ERROR: ОШИБКА:  команда CREATE TABLE отключена
CONTEXT:  функция PL/pgSQL abort_create_table_command(), строка 3, оператор RAISE
```

Требования к содержанию отчета

В отчет необходимо включить:

- Листинги SQL запросов для создания всех триггеров;
- Скриншоты результатов срабатывания всех триггеров;
- Скриншот правильной обработки исключения от триггера в приложении C#;
- выводы по работе.

Контрольные вопросы

1 Что такое триггер?

2 Какие бывают фазы срабатывания триггера?

3 Какова разница между триггерами различных фаз срабатывания?

4 Каково назначение DML-триггера?

5 Охарактеризовать события DML-триггера.

6 Каково назначение DDL-триггера?

7 Какие объекты доступны в триггерных функциях?